

Csharp_week4_Day_3_Hands_on_Gurunath_Rageni

Level-1 Problem 1: Student Grade Calculator

Scenario:

A school wants to calculate the average marks of a student using a class-based approach.

Requirements:

1. Create a class Student.
2. Create method CalculateAverage(int m1, int m2, int m3).
3. Return the average marks.
4. Display grade based on average.

Technical Constraints:

1. Use return type double for average.
2. Avoid hard-coded values.

Expectations:

Clear separation of logic inside methods.

Learning Outcome:

Learn method creation, return values, and basic OOP concepts.

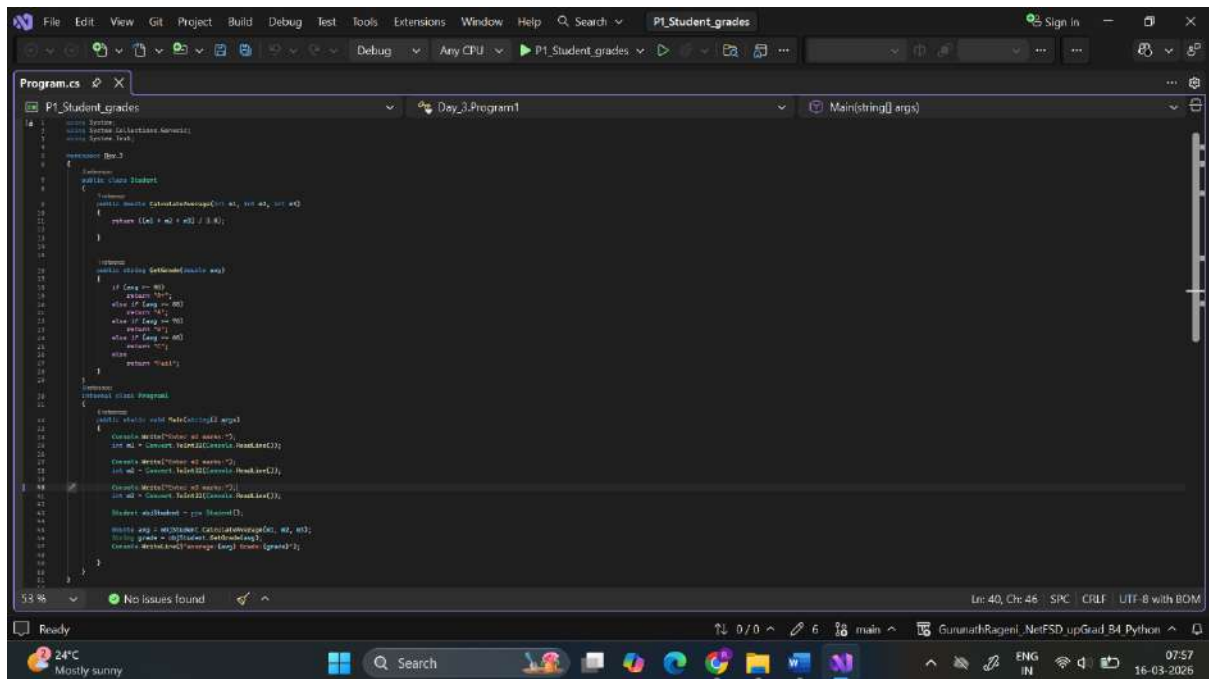
Sample Input:

80 70 90

Sample Output:

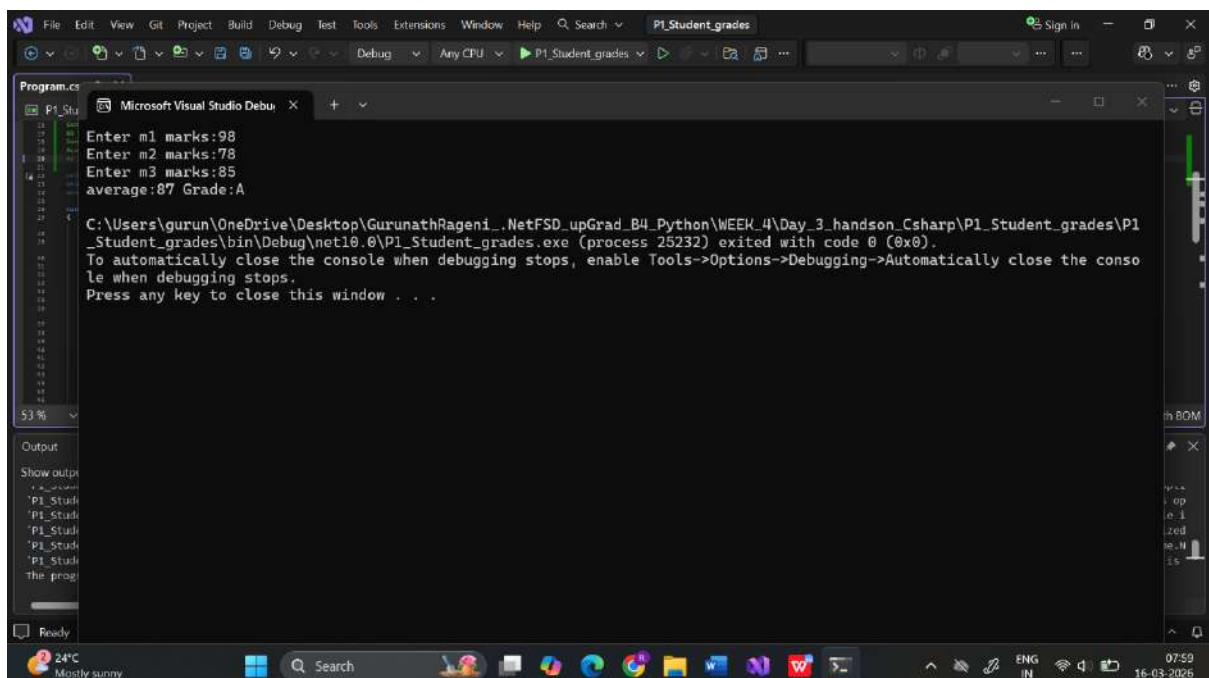
Average = 80, Grade = A

Code screenshot:



```
1 using System;
2 namespace P1_Student_grades
3 {
4     // Day 3
5     class Program
6     {
7         // Static method to calculate average of m1, m2, m3
8         static double CalculateAverage(int m1, int m2, int m3)
9         {
10             return (m1 + m2 + m3) / 3.0;
11         }
12
13         // Static method to display the average
14         static string GetGrade(double avg)
15         {
16             if (avg >= 85)
17                 return "A";
18             else if (avg >= 75)
19                 return "B";
20             else if (avg >= 65)
21                 return "C";
22             else if (avg >= 55)
23                 return "D";
24             else
25                 return "Fail";
26         }
27     }
28
29     // Main method
30     static void Main(string[] args)
31     {
32         Console.WriteLine("Enter m1 marks:");
33         int m1 = Convert.ToInt32(Console.ReadLine());
34
35         Console.WriteLine("Enter m2 marks:");
36         int m2 = Convert.ToInt32(Console.ReadLine());
37
38         Console.WriteLine("Enter m3 marks:");
39         int m3 = Convert.ToInt32(Console.ReadLine());
40
41         double avg = CalculateAverage(m1, m2, m3);
42         string grade = GetGrade(avg);
43         Console.WriteLine("Average: {0} Grade: {1}", avg, grade);
44     }
45 }
```

Output:



```
11 Enter m1 marks:98
12 Enter m2 marks:78
13 Enter m3 marks:85
14 average:87 Grade:A
15
16 C:\Users\gurun\OneDrive\Desktop\GurunathRagani..NetFSD.upGrad_B4_Python\WEEK_4\Day_3_handson_Csharp\P1_Student_grades\P1_Student_grades\bin\Debug\net10.0\P1_Student_grades.exe (process 25232) exited with code 0 (0x0).
17 To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
18 Press any key to close this window . . .
```

Level-2 Problem 1: Bank Account with Encapsulation

Scenario:

A bank wants to manage customer accounts securely using encapsulation.

Requirements:

1. Create class BankAccount.
2. Private field: balance.
3. Public methods: Deposit(double amount), Withdraw(double amount).
4. Method GetBalance() to return balance.
5. Prevent withdrawal if insufficient balance.

Technical Constraints:

1. Balance must be private.
2. Access balance only through public methods.
3. Use appropriate return types.

Expectations:

Proper use of encapsulation and object-oriented principles.

Learning Outcome:

Understand encapsulation, access modifiers, and secure data handling.

Sample Input:

Deposit 1000, Withdraw 300

Sample Output:

Current Balance = 700

Code screenshot:

```
Enter 1 for Deposit
Enter 2 for Withdraw
Enter 3 for Check Balance
Enter 4 for Exit
Enter Your Option:1
Enter amount for Deposit: 1000000
Remaining Balance: 1000000
Enter 1 for Deposit
Enter 2 for Withdraw
Enter 3 for Check Balance
Enter 4 for Exit
Enter Your Option:4
You have successfully exit
```

Level-2 Problem 2: Bank Account Management System

Scenario:

A bank wants to develop a simple console-based application to manage customer bank accounts. The system should protect account balance information and allow controlled access using properties.

Requirements:

1. Create a BankAccount class with private fields for account number and balance.
2. Use properties to allow controlled access to account number and balance.
3. Implement Deposit and Withdraw methods with proper validation.
4. Prevent withdrawal if balance is insufficient.

Technical Constraints:

- Use private fields with public properties.
- Apply encapsulation and data hiding.
- No direct access to balance field from outside the class.

Expectations:

- Demonstrate correct use of access modifiers.
- Validate negative deposit or withdrawal amounts.
- Display updated balance after each transaction.

Learning Outcome:

- Understand encapsulation using properties.
- Apply data hiding effectively.
- Implement validation logic inside class methods.

Sample Input:

Deposit = 5000, Withdraw = 2000

Sample Output:

Current Balance = 3000

Code screenshot:

```
C# P3_Banksystem_Encapsulation_datahiding
25 using System;
26 namespace BankApp
27 {
28     // reference
29     class BankAccount
30     {
31         // Private fields (data hiding)
32         private int accountNumber;
33         private double balance;
34
35         // Property for Account Number
36         // reference
37         public int AccountNumber
38         {
39             get { return accountNumber; }
40             set { accountNumber = value; }
41         }
42
43         // Property for Balance (read-only outside)
44         // reference
45         public double Balance
46         {
47             get { return balance; }
48         }
49
50         // reference
51         public void Deposit(double amount)
52         {
53             if (amount <= 0)
54             {
55                 Console.WriteLine("Invalid deposit amount");
56             }
57             else
58             {
59                 balance += amount;
60                 Console.WriteLine("Amount Deposited: " + amount);
61                 Console.WriteLine("Current Balance = " + balance);
62             }
63         }
64
65         // Withdraw Method
66         // reference
67         public void Withdraw(double amount)
68         {
69             if (amount <= 0)
70             {
71                 Console.WriteLine("Invalid withdrawal amount");
72             }
73             else if (amount > balance)
74             {
75                 Console.WriteLine("Insufficient Balance");
76             }
77             else
78             {
79                 balance -= amount;
80                 Console.WriteLine("Amount Withdrawn: " + amount);
81                 Console.WriteLine("Current Balance = " + balance);
82             }
83         }
84     }
85
86     // reference
87     class Program
88     {
89         // 0 references
90         static void Main(string[] args)
91         {
92             BankAccount account = new BankAccount();
93
94             account.AccountNumber = 181;
95
96             Console.Write("Enter Deposit Amount: ");
97             double deposit = Convert.ToDouble(Console.ReadLine());
98             account.Deposit(deposit);
99
100            Console.Write("Enter Withdraw Amount: ");
101            double withdraw = Convert.ToDouble(Console.ReadLine());
102            account.Withdraw(withdraw);
103
104            Console.WriteLine("Final Balance = " + account.Balance);
105        }
106    }
107 }
```

36 % No issues found

Output Screenshot:

```
Enter Deposit Amount: 5000000
Amount Deposited: 500000
Current Balance = 500000
Enter Withdraw Amount: 10000
Amount Withdrawn: 10000
Current Balance = 490000
Final Balance = 490000
```

Level-2 Problem 3: Employee Salary Calculator

Scenario:

A company wants to calculate employee salaries using inheritance. All employees have a base salary, but different roles calculate bonuses differently.

Requirements:

1. Create a base class Employee with Name and BaseSalary properties.
2. Create derived classes Manager and Developer.
3. Override a virtual method CalculateSalary().
4. Manager gets 20% bonus, Developer gets 10% bonus.

Technical Constraints:

- Use inheritance and method overriding.
- Apply polymorphism using base class reference.
- Use properties for salary details.

Expectations:

- Demonstrate runtime polymorphism.
- Avoid code duplication.
- Display final calculated salary.

Learning Outcome:

- Understand inheritance hierarchy.

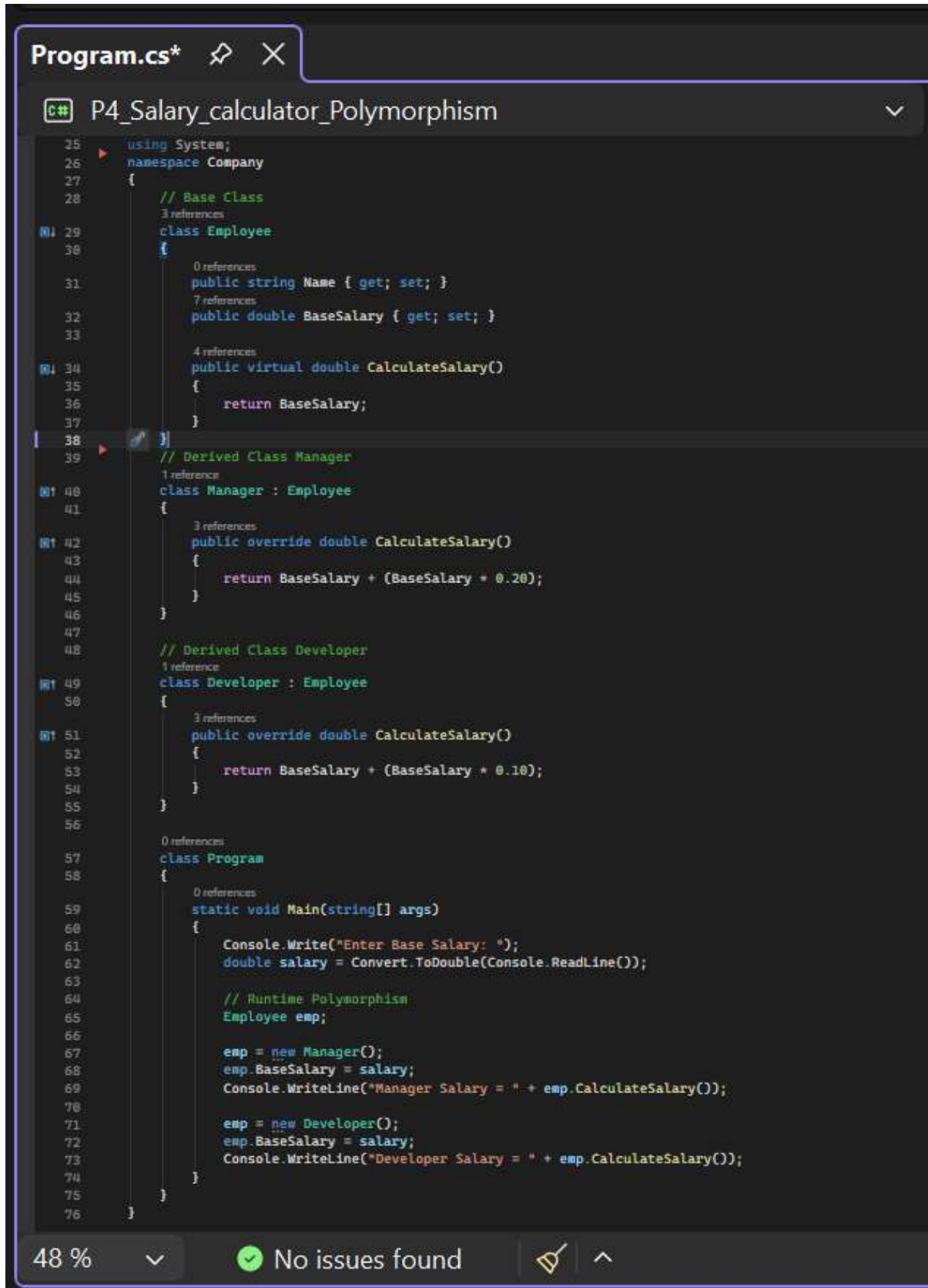
Sample Input:

BaseSalary = 50000

Sample Output:

Manager Salary = 60000, Developer Salary = 55000

Code Screenshot:



```
Program.cs*
P4_Salary_calculator_Polymorphism

25 using System;
26 namespace Company
27 {
28     // Base Class
29     class Employee
30     {
31         public string Name { get; set; }
32         public double BaseSalary { get; set; }
33
34         public virtual double CalculateSalary()
35         {
36             return BaseSalary;
37         }
38     }
39     // Derived Class Manager
40     class Manager : Employee
41     {
42         public override double CalculateSalary()
43         {
44             return BaseSalary + (BaseSalary * 0.20);
45         }
46     }
47
48     // Derived Class Developer
49     class Developer : Employee
50     {
51         public override double CalculateSalary()
52         {
53             return BaseSalary + (BaseSalary * 0.10);
54         }
55     }
56
57     class Program
58     {
59         static void Main(string[] args)
60         {
61             Console.WriteLine("Enter Base Salary: ");
62             double salary = Convert.ToDouble(Console.ReadLine());
63
64             // Runtime Polymorphism
65             Employee emp;
66
67             emp = new Manager();
68             emp.BaseSalary = salary;
69             Console.WriteLine("Manager Salary = " + emp.CalculateSalary());
70
71             emp = new Developer();
72             emp.BaseSalary = salary;
73             Console.WriteLine("Developer Salary = " + emp.CalculateSalary());
74         }
75     }
76 }
```

48 % No issues found

Output Screenshot:

```
Enter Base Salary: 50000
Manager Salary = 60000
Developer Salary = 55000
```

Level-2 Problem 4: Online Shopping Cart System

Scenario:

An e-commerce platform needs a flexible cart system where different product types calculate discounts differently.

Requirements:

1. Create a base class Product with properties Name and Price.
2. Create derived classes Electronics and Clothing.
3. Implement a virtual method CalculateDiscount().
4. Electronics get 5% discount, Clothing gets 15% discount.
5. Use encapsulation to protect price updates.

Technical Constraints:

- Use private fields with public properties.
- Apply inheritance and method overriding.
- Prevent negative price assignment.

Expectations:

- Demonstrate polymorphic behavior in cart processing.
- Apply data validation inside properties.
- Calculate and display final price after discount.

Learning Outcome:

- Combine encapsulation and polymorphism.
- Design extensible product hierarchy.
- Implement business logic in overridden methods.

Sample Input: Electronics Price = 20000

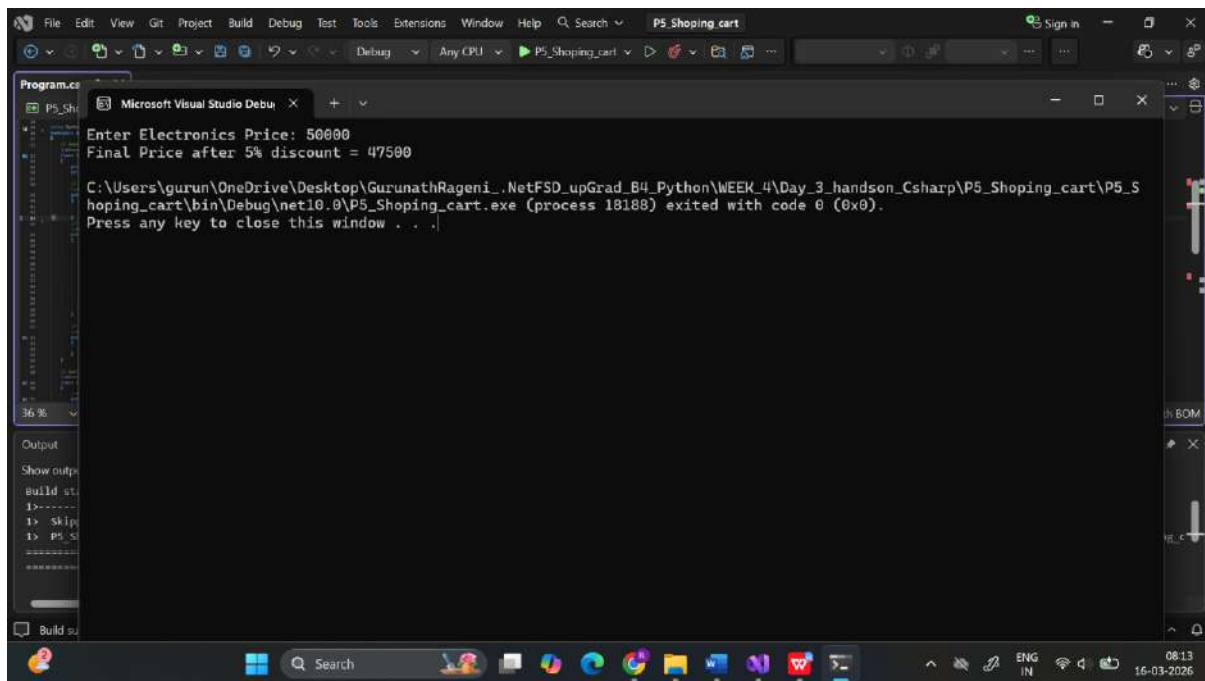
Sample Output: Final Price after 5% discount = 19000

Code Screenshot:

```
Program.cs*  
C# P5_Shopping_cart  
28
29 using System;
30 namespace ShoppingCart
31 {
32     // Base Class
33     3 references
34     class Product
35     {
36         private string name;
37         private double price;
38
39         // Property for Name
40         0 references
41         public string Name
42         {
43             get { return name; }
44             set { name = value; }
45         }
46
47         // Property for Price with validation
48         3 references
49         public double Price
50         {
51             get { return price; }
52             set
53             {
54                 if (value < 0)
55                 {
56                     Console.WriteLine("Price cannot be negative");
57                 }
58                 else
59                 {
60                     price = value;
61                 }
62             }
63         }
64
65         // Virtual Method
66         3 references
67         public virtual double CalculateDiscount()
68         {
69             return price;
70         }
71
72         // Derived Class Electronics
73         1 reference
74         class Electronics : Product
75         {
76             2 references
77             public override double CalculateDiscount()
78             {
79                 return Price - (Price * 0.05);
80             }
81         }
82
83         // Derived Class Clothing
84         0 references
85         class Clothing : Product
86         {
87             2 references
88             public override double CalculateDiscount()
89             {
90                 return Price - (Price * 0.15);
91             }
92         }
93
94         0 references
95         class Program
96         {
97             0 references
98             static void Main(string[] args)
99             {
100                 Console.Write("Enter Electronics Price: ");
101                 double price = Convert.ToDouble(Console.ReadLine());
102
103                 // Polymorphism
104                 Product product = new Electronics();
105                 product.Price = price;
106
107                 Console.WriteLine("Final Price after 5% discount = " + product.CalculateDiscount());
108             }
109         }
110     }
111 }
```

36 % 0 1

Output Screenshot:



```
Microsoft Visual Studio Debug Console Output:
Enter Electronics Price: 50000
Final Price after 5% discount = 47500
C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_3_handson_Csharp\P5_Shopping_cart\P5_S
hoping_cart\bin\Debug\net10.0\P5_Shopping_cart.exe (process 18188) exited with code 0 (0x0).
Press any key to close this window . . .
```

Level-2 Problem 5: Vehicle Rental System

Scenario:

A vehicle rental company wants a system where different vehicle types calculate rental charges differently.

Requirements:

1. Create a base class Vehicle with properties Brand and RentalRatePerDay.
2. Create derived classes Car and Bike.
3. Override CalculateRental(int days) method.
4. Car adds insurance charge of 500 per rental.
5. Bike offers 5% discount on total rental.

Technical Constraints:

- Use encapsulation with proper access modifiers.
- Apply runtime polymorphism.
- Validate number of rental days.

Expectations:

- Use base class reference to call overridden methods.
- Implement clean class hierarchy.
- Display final rental cost.

Learning Outcome:

- Master inheritance and polymorphism.
- Implement real-world OOP scenarios.
- Improve object-oriented design skills.

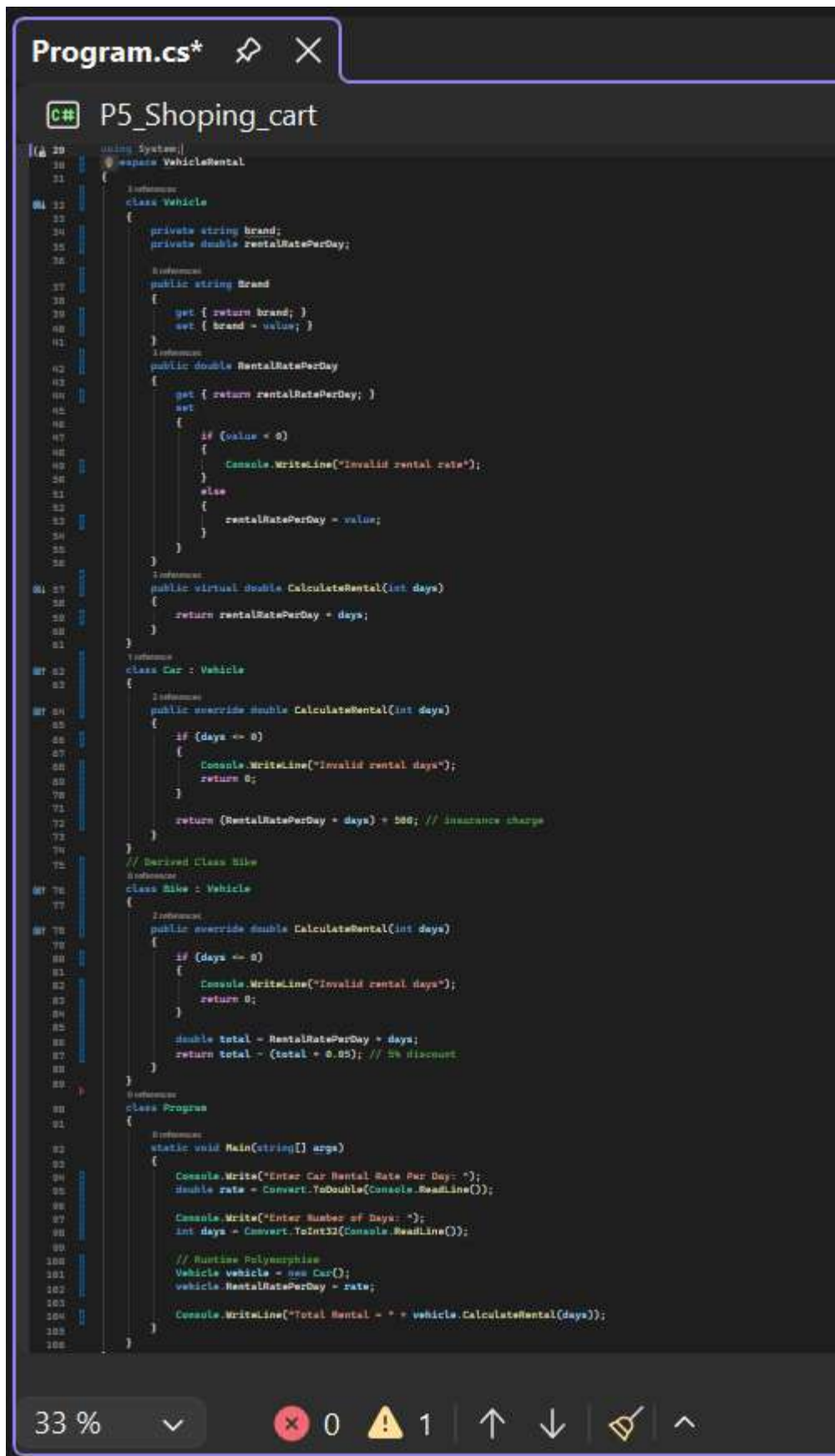
Sample Input:

Car RentalRatePerDay = 2000, Days = 3

Sample Output:

Total Rental = 6500

Code Screenshot:



```
Program.cs*
C# P5_Shopping_cart

using System;

namespace VehicleRental
{
    // Base Class
    class Vehicle
    {
        private string brand;
        private double rentalRatePerDay;

        // Properties
        public string Brand
        {
            get { return brand; }
            set { brand = value; }
        }

        public double RentalRatePerDay
        {
            get { return rentalRatePerDay; }
            set
            {
                if (value < 0)
                {
                    Console.WriteLine("Invalid rental rate");
                }
                else
                {
                    rentalRatePerDay = value;
                }
            }
        }

        // Method
        public virtual double CalculateRental(int days)
        {
            return rentalRatePerDay * days;
        }
    }

    // Derived Class Car
    class Car : Vehicle
    {
        // Method
        public override double CalculateRental(int days)
        {
            if (days <= 0)
            {
                Console.WriteLine("Invalid rental days");
                return 0;
            }

            return (RentalRatePerDay * days) + 500; // Insurance charge
        }
    }

    // Derived Class Bike
    class Bike : Vehicle
    {
        // Method
        public override double CalculateRental(int days)
        {
            if (days <= 0)
            {
                Console.WriteLine("Invalid rental days");
                return 0;
            }

            double total = RentalRatePerDay * days;
            return total - (total * 0.05); // 5% discount
        }
    }

    // Program Class
    class Program
    {
        // Main Method
        static void Main(string[] args)
        {
            Console.WriteLine("Enter Car Rental Rate Per Day: ");
            double rate = Convert.ToDouble(Console.ReadLine());

            Console.WriteLine("Enter Number of Days: ");
            int days = Convert.ToInt32(Console.ReadLine());

            // Runtime Polymorphism
            Vehicle vehicle = new Car();
            vehicle.RentalRatePerDay = rate;

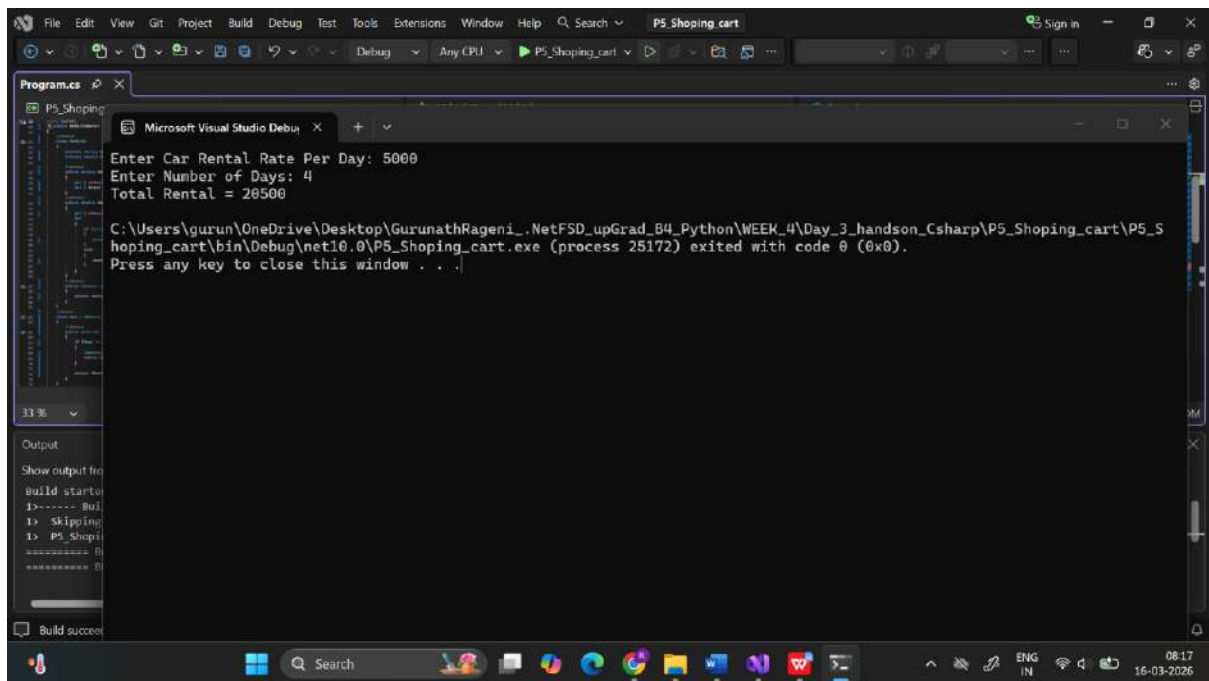
            Console.WriteLine("Total Rental = " + vehicle.CalculateRental(days));
        }
    }
}
```

33 %

0 1

↑ ↓

Output Screenshot:



```
Microsoft Visual Studio Debug Console
Enter Car Rental Rate Per Day: 5000
Enter Number of Days: 4
Total Rental = 28500
C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_3_handson_Csharp\P5_Shoping_cart\P5_S
hoping_cart\bin\Debug\net10.0\P5_Shoping_cart.exe (process 25172) exited with code 0 (0x0).
Press any key to close this window . . .
```

Output

Show output from build started

1>----- Build started

1> Skipping

1> P5_Shoping_cart

Build succeeded